

DEPLOYMENT PLAN

List of Contributors

Hatice Melike Aydın 231101063

İclal Zeynep Kaya 231101073

Anıl Özişler 211101081

Task Matrix

Hatice Melike Aydın	İclal Zeynep Kaya	Anıl Özişler
Deployment Overview	Deployment Process	Configuration Plan

Table Of Contents

1. Deployment Overview

- Frontend Hosting
- Backend Service
- Code Repository
- Scalability & Maintenance

2. Deployment Process

- Push Latest Code to GitHub Repository
- Frontend Deployment via Firebase Hosting
- Firebase Authentication Setup
- Database Configuration
- API Key Management & Security
- Monitoring and Maintenance

3. Configuration Plan

- Frontend Deployment
- Authentication
- Data Services(Matching/Location)
- Database
- Monitoring
- Security

○ Deployment Overview

Synkly is deployed as a single-page web application (SPA) using Firebase Hosting for the frontend and Google Firebase services for all backend and database functionality.

1. **Frontend Hosting:**

- Hosted on Firebase Hosting to ensure fast content delivery and integration with the primary technology stack.
- The application is built using React and JavaScript.

2. **Backend Service:**

- Firebase Authentication handles secure user sign-up, login, and session management.
- Firestore Database stores all application data, including user profiles, class schedules, and match results.
- The system uses the internal Matching Service and Location Service functionalities.

3. **Code Repository:**

- Source code is managed on GitHub.
- Deployment process will leverage the GitHub repository for continuous updates.

4. **Scalability & Maintenance:**

- Firebase (Authentication and Firestore) provides a highly scalable and serverless infrastructure, requiring minimal maintenance for the backend.

○ Deployment Process

1. Push Latest Code to GitHub Repository

- Ensure all latest code, including the React components and Firebase integration scripts, is committed and pushed to the main branch of the GitHub repository.
- `git add .`
- `git commit -m "Final deployment commit"`
- `git push origin main`

2. Frontend Deployment via Firebase Hosting

- Install the Firebase CLI.
- Run `firebase init` in the project root to configure Firebase Hosting.
- Run `npm run build` to compile the React application for production.
- Run `firebase deploy only hosting` to deploy the application to the live URL.

3. Firebase Authentication Setup

- In the Firebase Console, navigate to the Authentication section.
- Enable Email/Password sign-in providers to support user registration and login.
- Configure Firebase SDK within the application (`firebase.js`) (`Register.js`) (`Login.js`) to handle authentication functions.

4. Database Configuration

- Enable Firestore in Firebase Console.
- Define the necessary collections: `users`, `schedules`, and `invitations`.
- Implement Security Rules for Firestore to ensure that users can only read/write their own schedule data, protecting data privacy.

5. Key Management & Security

- Since Synkly primarily uses Firebase services, the configuration details should be treated as sensitive data.
- Store Firebase configuration keys securely using environment variables during the build and deployment process.
- HTTPS is automatically enforced by Firebase Hosting for secure communication.

6. Monitoring and Maintenance

- Monitor Firebase usage, database activity, and authentication logs via the Firebase Console to track system performance and user interactions.
- Regularly review Firestore security rules to ensure continued data integrity and access control.

○ Configuration Plan

Frontend Deployment	Hosted on Firebase Hosting for easy access and integration with the existing tech stack. The React.js framework is used to build a modular and responsive user interface.
Authentication	Firebase Authentication is used to handle secure user sign-ups and logins via e-mail/password. Firebase rules ensure that unauthorized access is blocked.
Data Services (Matching/Location)	The Matching Service relies entirely on schedule data stored in Firestore. Location data for campus dining places is stored securely.
Database	Firestore Database stores all critical data: users, schedules, invitations, and chats. Security rules are implemented to control data access, ensuring only authenticated users can access and modify their own data
Monitoring	Currently, manual monitoring is performed by checking user activity, data integrity, and error logs directly through the Firebase Console.
Security	All sensitive configuration data is stored securely using environment variables. Firebase security rules ensure that only authorized users can read or write data to the database, enforcing strict access control for schedules and invitations.